

angular-inport

Audit date - 2026-05-08 18:25

Angular ^21.2.6 - 30 TS files - 2,376 LOC

51/100

Grade D

Faible - problemes significatifs, refactoring urgent conseille.

48 issues detected

0

CRITICAL

43

IMPORTANT

5

MINOR

Project overview

Angular version	^21.2.6
TypeScript files	30
HTML templates	5
Components / Services / Modules	5 / 0 / 3
Pipes / Guards	0 / 0
Total lines of code	2,376
Tests detected	Yes

Severity summary



Critical: 0

Important: 43

Minor: 5

Issues detail

Type Safety

IMPORTANT

TYPE001 Usage de 'any' TypeScript (40)

Le type `any` désactive TypeScript. Cache des bugs, rend le refactoring dangereux.

Fix: Typer explicitement (interface, type, générique). Utiliser `unknown` si le type est vraiment inconnu.

OCCURRENCES

[projects/angular-inport/src/lib/inview-container.directive.spec.ts:65](#)

```
lastEvent: any;
```

[projects/angular-inport/src/lib/inview-container.directive.spec.ts:66](#)

```
onInview(event: any) { this.lastEvent = event; }
```

[projects/angular-inport/src/lib/inview-container.directive.spec.ts:79](#)

```
lastEvent: any;
```

[projects/angular-inport/src/lib/inview-container.directive.spec.ts:80](#)

```
onInview(event: any) { this.lastEvent = event; }
```

[projects/angular-inport/src/lib/inview-container.directive.spec.ts:88](#)

```
lastEvent: any;
```

[projects/angular-inport/src/lib/inview-container.directive.spec.ts:89](#)

```
onInview(event: any) { this.lastEvent = event; }
```

[projects/angular-inport/src/lib/inview-container.directive.spec.ts:94](#)

```
async function setupModule(hostType: any): Promise<{
```

[projects/angular-inport/src/lib/inview-container.directive.spec.ts:96](#)

```
host: any;
```

... and 32 more occurrence(s).

Memory Leaks

IMPORTANT

JS001 setTimeout/setInterval sans cleanup (1)

Un `setTimeout` ou surtout `setInterval` lance dans un composant qui n'est jamais clear continue a tourner apres la destruction du composant. Sur une SPA Angular, accumuler des intervalles oubliés = memory leak progressif + appels reseau fantomes. Different de RxJS subscriptions (gere par MEM001).

Fix: Garder la reference (`this.timerId = setTimeout(...)`) et appeler `clearTimeout(this.timerId)` dans `ngOnDestroy()`. Ou

mieux : utiliser ``interval(N).pipe(takeUntilDestroyed())`` (Angular 16+) qui s'auto-nettoie.

OCCURRENCES

[projects/angular-inport-example/src/app/pages/simple.component.ts:29](#)

```
setTimeout(() => (this.ready = true));
```

Performance

IMPORTANT **PERF002 Route sans lazy loading (2)**

Les routes chargées *eagerly* augmentent le bundle initial et ralentissent le démarrage.

Fix: Remplacer ``component: MyComponent`` par ``loadComponent: () => import('./my.component').then(m => m.MyComponent)``

OCCURRENCES

[projects/angular-inport-example/src/app/app-routing.module.ts:8](#)

```
{ path: 'simple', component: SimpleComponent },
```

[projects/angular-inport-example/src/app/app-routing.module.ts:9](#)

```
{ path: 'benchmark', component: BenchmarkComponent },
```

Code Quality

MINOR **DEBUG001 console.log en production (5)**

Les `console.log` oubliés exposent des données internes en prod et polluent la console.

Fix: Supprimer ou remplacer par un service de logging. Configurer ``build.optimization.scripts`` pour stripper en prod.

OCCURRENCES

[projects/angular-inport/src/lib/benchmark.spec.ts:86](#)

```
console.log(`[BENCH] ${label}: ${ops} ops in ${durationMs.toFixed(1)} ms ? ${perOp} ms/op`);
```

[projects/angular-inport/src/lib/benchmark.spec.ts:212](#)

```
console.log(`[BENCH] zone.run() calls for ${n} items x ${ROUNDS} rounds: ${zoneRunCount} (expec...
```

[projects/angular-inport/src/lib/benchmark.spec.ts:256](#)

```
console.log(`[BENCH] ${CYCLES} create/destroy cycles completed without error`);
```

[projects/angular-inport/src/lib/benchmark.spec.ts:296](#)

```
console.log(`[BENCH] tooLazy: zone.run() called ${zoneRunAfterFirst} times for ${ROUNDS} duplic...
```

[projects/angular-inport-example/src/main.ts:13](#)

```
.catch((err) => console.error(err));
```

Lazy loading

Eager routes (no lazy)	2
Lazy routes	0
Lazy loading ratio	0%

Less than 50% of routes use lazy loading. Each eager route adds to the initial bundle. Migrate to `loadComponent` (Angular 15+) for the heaviest routes first.

Refactoring plan

This month - Important

- Usage de 'any' TypeScript (TYPE001)

Le type `any` désactive TypeScript. Cache des bugs, rend le refactoring dangereux.

- setTimeout/setInterval sans cleanup (JS001)

Un `setTimeout` ou surtout `setInterval` lance dans un composant qui n'est jamais clear continue a tourner apres la destruction...

- Route sans lazy loading (PERF002)

Les routes chargées eagerly augmentent le bundle initial et ralentissent le démarrage.

On the roadmap - Minor

- console.log en production (DEBUG001)

Les console.log oubliés exposent des données internes en prod et polluent la console.

This report is produced by automated static analysis. It does not replace a thorough manual review by an experienced Angular developer.